功能结构设计（高校考场编排与监考分配系统 v8）

> 2026-11-02。指导教师认为「健身 / 健康管理」类题材重复度高，本项目更换题材为**高校考场编排与监考分配**。
>
> 前七版（生活数据、三角色管理平台、健身、社交化）全部作废，仅保留其框架层代码。本文件是新领域的完整设计口径。
>
> **技术核心是排考算法，不是增删改查**——这是它与前几版「管理系统」的根本区别。

1. 选题定位

高校期末考试周的排考，本质是一个**多约束组合优化问题**：把「考试」安排到「时段 × 教室」，再把「监考教师」匹配到每一场，同时满足学生不冲突、教室容量、教师回避、监考均衡等一堆条件。

教务人员目前的实际做法是 Excel 加人工经验：300 门课排一次要一周，冲突靠事后核对，出了差错只能局部返工，而且没法回答「这个方案比上一个好在哪」。

系统要解决三件事：

问题	现状	系统做法
约束多且互相牵制	人工只能顾住最显眼的几条（如学生不冲突），回避与负载均衡常被忽略	显式建模硬约束与软约束，逐条校验并定位
手工排不动	规模一大就靠局部返工，越改越乱	分阶段求解：着色 → 装箱 → 匹配 → 局部搜索
无从评价优劣	排完就发，没有质量指标	定义冲突数、软成本、工作量标准差、教室利用率并出一份报告

2. 业务流程

```
基础数据（学期 / 课程 / 教学班 / 学生 / 教师 / 教室 / 时段）
        ↓
考试计划申报与审核（课程 → 考试时长、期望时段、形式）
        ↓
资源与约束配置（教师可用时段、回避申报、教室停用）
        ↓
自动编排 → 生成方案（可多版本）→ 冲突检测 → 手动调整
        ↓
座位编排（按学号 / 随机）→ 监考分配与负载均衡
        ↓
结果发布 → 学生查询 / 教师监考通知
        ↓
变更（请假 → 替监 → 局部重排）→ 考场情况上报 → 统计报表
```

考试状态机：`DRAFT`（申报）→ `APPROVED`（审核通过）→ `SCHEDULING`（编排中）→ `SCHEDULED`（已编排）→ `PUBLISHED`（已发布）→ `CHANGED`（变更中）。

3. 核心算法设计

这一节是整个选题的技术支撑，也是论文第 4 章的内容。

3.1 问题建模

输入

- 考试集合 `E = {e1…en}`，每场已知：所属教学班、考生集合、时长、可接受时段范围
- 时段集合 `T`（考试周内每天 3 个时段 × 天数）
- 教室集合 `R`，每间有容量与类型；部分教室在部分时段不可用
- 教师集合 `P`，含监考资格、可用时段、回避关系

冲突图

以考试为顶点，若两场考试存在共同考生（同班或同学分制班级），则连边。排考的第一步就是给这个图着色，颜色即时段。

硬约束

编号	约束
C1	同一考生不能在同一时段参加两场考试
C2	同一教室同一时段只能安排一场考试
C3	一场考试的考生总数不得超过所分配教室的容量之和
C4	同一教师同一时段不能监考两场
C5	教师回避的课程或班级不得安排其监考
C6	教师未申报可用的时段不得安排其监考
C7	考试时长不得超过时段长度

软约束（目标函数里加权）

编号	约束	权重意义
S1	监考工作量均衡（各教师场次的标准差）	公平性
S2	同一教师避免连续多个时段监考	疲劳
S3	教室利用率（容量浪费最小）	资源
S4	学生同一考试日的场次尽量不超 2 场	学生体验

目标函数

```
min f = M × Σ硬违约 + Σ wi × 软成本i
```

其中 `M` 取一个远大于软成本总和的大数，保证任何硬违约都不可接受。

3.2 分阶段求解

阶段	任务	方法	复杂度
1	时间分配	按冲突度降序的 **DSatur 图着色**	$O(n^2)$
2	教室分配	**首次适应递减（FFD）装箱**，一场考试可拆多间教室	O(n log n)
3	监考分配	**贪心 + 负载再平衡**：优先可用、无回避、当前负载最低的教师	O(n·m)
4	冲突修复	**模拟退火**：邻域为「移动考试时段」「交换两场时段」「更换教室」	可配置迭代数

分阶段而不是一次性求解，是工程上的取舍：全局搜索（如遗传算法）质量可能更好，但不可控、难调试、给教师的解释成本高。分阶段每一步都有明确语义，出了问题能定位到「哪一场、哪一条约束」。

3.3 评价指标

指标	含义
硬冲突数	必须为 0 才能发布
软成本	加权后的 S1–S4 之和
监考工作量极差 / 标准差	越小越均衡
教室利用率	实际使用座位 / 分配教室总容量
求解耗时	用于说明可交互性

3.4 实验设计（第 6 章素材）

- 规模：100 / 300 / 500 场考试（按本校实际规模设定，参数可配置）
- 对比：纯贪心 · DSatur · DSatur + 装箱 · 全流程（DSatur + FFD + SA）
- 指标：冲突数、软成本、耗时、工作量标准差
- 数据：**模拟生成器**（冲突率、教室容量分布、教师回避比例可调），并说明生成规则与局限性

4. 角色与端

角色	主端	职责	关键权限
教务管理员	Web 后台	基础数据、考试计划审核、编排与调整、发布、统计	全部编排操作；可查看冲突与质量报告
教师	Web	查看监考、申报可用时段与回避、请假与替监、签到、情况上报	只看自己的监考；不能改他人安排
学生	Android（uni-app）	查看考试安排、考场座位、变更通知、缓考申请	只看本人考试

角色比前几版更「薄」，但这正是该类系统的真实形态：**系统的重量在算法与管理端，学生端天然是只读的。**工作量不靠学生端堆模块，而靠编排引擎与教务流程。

5. 功能结构（36 个）

5.1 教务管理员（20 个）

编号	模块	优先级	说明
A1	数据看板	P0	考试总数、已编排比例、冲突数、教室占用率、监考负载概览
A2	学期管理	P0	学年学期、考试周起止、每天时段数
A3	课程管理	P0	课程号、名称、学分、考核方式
A4	班级管理	P0	年级、专业、人数、所属院系
A5	学生管理	P0	学号、姓名、班级；批量导入
A6	教师管理	P0	工号、院系、监考资格、可用时段、回避
A7	教室管理	P0	楼栋、容量、类型、停用时段
A8	时段管理	P0	考试周内具体时段（日期 + 上午/下午/晚上）
A9	考试计划	P0	课程申报考试、时长、期望时段；审核通过进入待编排
A10	排考规则	P0	硬软约束开关与权重、模拟退火参数
A11	自动编排	P0	**核心**：触发分阶段求解，生成方案版本，展示过程指标
A12	手动调整	P0	拖拽换时段/教室，实时校验冲突
A13	冲突检测	P0	全量校验、冲突定位（哪场、哪条约束、涉及谁）
A14	座位编排	P1	按学号、随机或打散策略生成座位号
A15	结果发布	P0	发布方案，推送教师与学生通知
A16	监考调整	P1	审批请假、安排替监、触发局部重排
A17	统计报表	P1	监考工作量、教室利用率、考试日程表导出
A18	数据导入	P1	课程/学生/教师/教室 Excel 批量导入与校验
A19	操作审计	P1	编排、发布、调整留痕
A20	系统设置	P0	参数、会话清理、通知开关

5.2 教师（9 个）

编号	模块	优先级	说明
T1	我的监考	P0	列表与日历视图：时间、考场、考生数
T2	可用时段	P0	申报可监考时段，影响分配
T3	回避申报	P0	回避某课程或班级，原因必填

编号	模块	优先级	说明
T4	请假申请	P1	发起请假，等待管理员审批与替监安排
T5	替监接手	P1	接替他人监考（需管理员确认）
T6	监考签到	P1	开考前签到，记录时间
T7	情况上报	P1	缺考、违纪、试卷异常上报
T8	通知中心	P0	编排结果、调换、系统通知
T9	个人中心	P0	资料、改密

5.3 学生（7 个）

编号	模块	优先级	说明
S1	我的考试	P0	课程、时间、考场、座位号
S2	考试日历	P0	按考试周展示
S3	考场座位	P0	考场与座位号，含楼栋楼层指引
S4	变更通知	P1	时间地点变更提醒
S5	缓考申请	P2	提交缓考理由与材料
S6	违纪查询	P2	违纪记录与处理结果
S7	个人中心	P0	资料、改密

6. 数据模型（37 张）

域	表	说明
账号与权限（7）	`app_user` `auth_session` `role` `user_role` `permission` `role_permission` `user_profile`	复用现有鉴权，增加角色区分
组织与人员（4）	`department` `class_info` `student_profile` `teacher_profile`	院系、班级、学生、教师（含监考资格）
课程与教学（3）	`course` `teaching_class` `teaching_class_student`	教学班是冲突判定与容量的基本单位
考试与资源（6）	`semester` `exam_period` `time_slot` `classroom` `classroom_unavailable` `exam`	时段与教室可达性
编排（5）	`schedule_plan` `exam_schedule` `exam_room` `seat_assignment` `invigilation_assignment`	方案版本、考试→时段、考试→教室（可多间）、座位、监考

域	表	说明
规则（3）	`scheduling_rule` `teacher_availability` `teacher_avoidance`	约束参数与教师申报
协同流程（4）	`leave_request` `substitute_request` `exam_incident` `makeup_exam`	请假/替监、现场上报、缓考
通知与后台（5）	`notification` `operation_log` `sys_config` `import_record` `announcement`	

两个关键设计：

1. `exam_room` **单独成表**，允许一场考试拆成多间教室（人数超单间容量），`exam_schedule` 只管考试→时段。这样「拆考场」不会污染编排主表。
2. `schedule_plan` **存方案版本**，每次自动编排产生一个版本，保留参数快照与质量指标，支持「方案 A 与方案 B 对比」，也支持回滚。

7. 技术路线与边界

层	选型
Android	uni-app（Vue 3），学生端为主
Web	Vue 3 + Vite，教务端与教师端
服务端	Java 17 + Spring Boot 4.1，`JdbcTemplate`
数据库	MySQL 8.4
认证	自研令牌 + 会话表（复用现有实现）
编排	服务端 Java 实现，异步任务 + 进度查询

明确不做：遗传算法调优平台、与教务系统对接、座位图可视化编辑、移动端排考、云部署、外部地图与短信服务。数据一律本地，导入靠 Excel。

8. 现有资产复用与迁移

现有资产	处理
登录鉴权（令牌摘要、会话上限、过滤器）	**复用**，加角色判断
角色/权限表设计	**复用**
后台骨架、路由、请求封装、界面设计系统	**复用**
启动脚本、测试骨架、部署说明	**复用**
记账、日志、身体、训练、社区相关代码	**删除**
已建 15 张表	备份后清理，新 schema 从 `09` 起编号

可复用部分约三到四成，主要是与业务无关的框架层。

9. 分期

阶段	时间	交付
定题	11.09 前	本文件 + 功能结构图 + 题目，交教师确认
开题前	11.17–12.28	账号与角色、基础数据六个模块（学期/课程/班级/学生/教师/教室/时段）、考试计划、排考规则；**开题报告含算法方案**
中期前	12.29–第八学期第 6 周	自动编排（着色 + 装箱 + 监考）、冲突检测、手动调整、结果发布；教师端全部；学生端查询
收尾	第 7 周后	座位编排、统计报表、模拟退火优化、实验评估、论文定稿

10. 论文结构

章	内容
第 1 章	绪论：排考问题的规模与痛点，约束优化与图着色研究现状
第 2 章	需求分析：三类角色、约束清单、非功能需求
第 3 章	系统设计：总体架构、数据模型（37 表）、模块划分
第 4 章	**排考算法设计与实现**：建模、四阶段求解、冲突检测与修复
第 5 章	系统实现：教务端、教师端、学生端、异步任务
第 6 章	实验与评估：规模实验、策略对比、指标分析
第 7 章	总结与展望

重点与难点（有代码和实验支撑）：约束建模的完整性（拆分考场、教师回避与可用时段）、分阶段求解的工程化、冲突的可定位与可解释。

11. 答辩口径

问：排课排考是被研究烂了的经典问题，你的创新点在哪？

> 不承诺算法理论创新。价值在三处工程实现：一是约束建模完整（含教师回避、可用时段、一场考试拆多间教室，多数简化系统不做）；二是分阶段求解可定位可解释，出问题能指出是哪一场、哪条约束；三是给出量化指标体系与策略对比实验。本科阶段把这三件事做扎实，比声称一个改进算法更实在。

问：数据从哪来？

> 模拟生成器，按本校实际规模设定参数（考试数、班级分布、教室容量、教师回避比例），生成规则写进论文，并说明局限：模拟数据无法完全反映真实教务的异常情况。

问：为什么不用遗传算法一步到位？

> 已做对比：全局搜索在软成本上可能更优，但不可控、调试成本高、给教务人员的解释成本高。分阶段方案每一步语义清晰，且能在 30 秒内出结果，可交互调整。实验章节会给出对比数据。

问：教师端和学生端功能很薄，是不是凑数？

> 这类系统的重量在编排引擎与管理端，学生端天然只读。模块数量不是目标，编排质量才是。

12. 待教师确认

1. **题目**：《基于约束满足的高校考场编排与监考分配系统的设计与实现》。
2. 该题材是否可接受（比「健身 / 健康」重复度低，但排课类仍需确认）。
3. 数据使用模拟生成是否可接受。
4. 现有业务代码作废、仅复用框架层，是否可以。